DBMS

Complete Language & Concepts Notes

From Basics to Advanced

A structured study guide covering database fundamentals, data models, ER diagrams, relational algebra, SQL, normalization, transactions, concurrency, indexing, recovery, distributed databases, NoSQL, security, and practical exercises.

For: students, beginners, interview preparation, and developers building database-backed applications.

How to use: learn the concepts, run SQL examples in a practice database, draw schemas, and solve the exercises.

SQL dialects vary across PostgreSQL, MySQL, SQL Server, Oracle, and SQLite. Examples use broadly familiar SQL; adjust syntax for your database.

Contents

1. Introduction to DBMS
2. Database architecture and data independence
3. Data models and schemas
4. ER model and ER diagrams
5. Relational model and keys
6. Relational algebra and calculus
7. SQL overview and data definition
8. SQL data manipulation and querying
9. Joins, subqueries, and set operations
10. Constraints, views, and indexes
11. Functional dependencies and normalization
12. Transactions and ACID
13. Concurrency control and locking
14. Deadlocks and schedules
15. Recovery and logging
16. Storage, files, and indexing structures
17. Query processing and optimization
18. Database security
19. Stored procedures, functions, and triggers
20. Distributed databases and replication
21. NoSQL databases
22. Data warehousing, OLAP, and analytics
23. Database design workflow
24. Performance and operational best practices
25. Common interview questions
26. Practice SQL schema and queries
27. Exercises and mini-projects
28. Quick revision sheet

1. Introduction to DBMS

A **database** is an organized collection of related data. A **Database Management System (DBMS)** is software that defines, stores, retrieves, updates, protects, and manages access to databases.

Why use a DBMS?

• Reduces unnecessary duplication and inconsistency.

• Supports concurrent access by multiple users.

• Enforces rules and constraints to improve data integrity.

• Provides security, authorization, backup, and recovery.

• Offers query languages such as SQL.

• Separates application logic from physical data storage.

DBMS versus file system

File-based approach	DBMS approach
Applications manage formats and access separately	Centralized schema and management
Duplication and inconsistency can be harder to control	Constraints and normalization help manage redundancy
Concurrency and recovery are application-specific	Transactions and recovery mechanisms are available
Queries often require custom code	Declarative queries with SQL

Common database types

• Relational: tables and relationships, usually queried with SQL.

• Document: JSON-like documents.

• Key-value: values retrieved by keys.

• Wide-column: data organized by column families.

• Graph: nodes and relationships.

• Time-series: time-stamped observations.

2. Database architecture and data independence

Three-schema architecture

• External level: user-specific views of data.

• Conceptual level: the logical structure of the whole database.

• Internal level: physical storage details, files, pages, and indexes.

Data independence means changes at one level can be made with limited impact on other levels. Physical data independence separates storage changes from the logical schema; logical data independence reduces the effect of logical schema changes on user views and applications.

DBMS components

Typical components include the query parser and optimizer, execution engine, storage manager, buffer manager, transaction manager, concurrency control, authorization system, and recovery manager.

Deployment patterns

A database may run locally, on a dedicated server, in a managed cloud service, or as part of a distributed system. Client-server architecture separates application clients from database services.

3. Data models and schemas

Term	Meaning
Schema	Planned structure and rules of a database
Instance/state	Data stored at a particular time
Relation/table	Set-like collection of rows with named columns
Tuple/row	One record
Attribute/column	A property of a relation
Domain	Allowed set of values for an attribute
Metadata	Data describing the database structure

A schema describes structure; an instance is the current data. A logical model explains entities and relationships, while a physical model specifies storage choices.

Data models

• Hierarchical: tree-shaped parent-child structure.

• Network: records connected by links.

• Relational: relations (tables) and constraints.

• Object-oriented: objects and object relationships.

• Document, key-value, column-family, and graph models for different workloads.

4. ER model and ER diagrams

Entity-Relationship (ER) modeling describes the real-world things a database stores before tables are designed.

Core concepts

• Entity: distinguishable object, such as Student or Course.

• Entity set: collection of entities of a type.

• Attribute: property such as StudentName.

• Relationship: association between entities.

• Key attribute: uniquely identifies an entity.

• Weak entity: depends on an owner entity for identification.

Cardinality and participation

Relationship	Example	Meaning
One-to-one (1:1)	Person—Passport	Each side relates to at most one on the other side
One-to-many (1:N)	Department—Employees	One department may have many employees
Many-to-many (M:N)	Students—Courses	Many students take many courses
Total participation	Every Order has a Customer	Every entity must participate
Partial participation	An Employee may manage no project	Participation is optional

Mapping ER to relational tables

• Strong entity becomes a table; its key becomes a primary key.

• For 1:N, place the foreign key on the N-side in the usual design.

• For M:N, create a junction table with foreign keys to both sides.

• A multivalued attribute usually becomes a separate table.

• A weak entity table includes its owner key plus a partial key.

```
STUDENT(StudentID PK, Name, Email)
COURSE(CourseID PK, Title)
ENROLLMENT(StudentID FK, CourseID FK, EnrolledOn,
           PRIMARY KEY (StudentID, CourseID))
```

5. Relational model and keys

The relational model represents data as relations. In practical SQL databases, tables can contain duplicate rows unless constraints prevent them, even though the formal relational model treats relations as sets.

Key type	Definition
Superkey	Any attribute set that uniquely identifies a row
Candidate key	Minimal superkey
Primary key	Candidate key selected as the main identifier
Alternate key	Candidate key not selected as primary
Foreign key	Attribute(s) referencing a key in another or same table
Composite key	Key containing multiple attributes
Surrogate key	Artificial identifier such as generated ID

Integrity constraints

• Domain integrity: values follow type and domain rules.

• Entity integrity: primary-key values are unique and not null.

• Referential integrity: foreign keys reference valid rows or permitted nulls.

• Business constraints: rules specific to the organization.

A primary key should be stable, unique, and non-null. Foreign keys help prevent orphan records and can define actions on update or delete.

6. Relational algebra and calculus

Relational algebra is a procedural formal foundation for relational queries; relational calculus is declarative and specifies what result is desired. SQL is influenced by both.

Operator	Symbol / idea	Meaning
Selection	σ condition (R)	Choose rows matching a predicate
Projection	π columns (R)	Choose attributes/columns
Union	R ∪ S	Rows in either compatible relation
Difference	R – S	Rows in R but not S
Cartesian product	R × S	Pair each row from R with each row from S
Join	R ■ S	Combine related rows
Rename	ρ	Rename relation or attributes

Union and difference require union-compatible relations. Selection filters rows; projection selects columns. Joins can be viewed as products followed by conditions. SQL SELECT clauses are not evaluated in the written order; conceptual processing often begins with FROM/JOIN, then WHERE, grouping, HAVING, SELECT, and ORDER BY.

7. SQL overview and data definition

SQL (Structured Query Language) is the standard family of languages for defining, querying, and modifying relational data. Exact features differ by database engine.

SQL command categories

Category	Purpose	Examples
DDL	Define structure	CREATE, ALTER, DROP, TRUNCATE
DML	Modify data	INSERT, UPDATE, DELETE, MERGE
DQL	Query data (common teaching label)	SELECT
DCL	Permissions	GRANT, REVOKE
TCL	Transactions	COMMIT, ROLLBACK, SAVEPOINT

```
CREATE TABLE Department (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(100) NOT NULL UNIQUE
);
```

```
CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    Salary DECIMAL(10,2) CHECK (Salary >= 0),
    DepartmentID INT,
    FOREIGN KEY (DepartmentID)
        REFERENCES Department(DepartmentID)
);
```

Choose suitable data types and constraints. VARCHAR limits and identity/auto-increment syntax vary by engine.

8. SQL data manipulation and querying

```
INSERT INTO Department (DepartmentID, DepartmentName)
VALUES (10, 'Engineering');

INSERT INTO Employee (EmployeeID, FullName, Salary, DepartmentID)
VALUES (1, 'Asha Rao', 65000.00, 10);

SELECT EmployeeID, FullName, Salary
FROM Employee
WHERE Salary >= 50000
ORDER BY Salary DESC;

UPDATE Employee
SET Salary = Salary * 1.05
WHERE EmployeeID = 1;

DELETE FROM Employee
WHERE EmployeeID = 1;
```

Filtering, sorting, and NULL

```
SELECT DISTINCT DepartmentID
FROM Employee
WHERE Salary BETWEEN 40000 AND 80000
  AND FullName LIKE 'A%';

SELECT *
FROM Employee
WHERE DepartmentID IS NULL;
```

NULL means missing/unknown/not applicable; it is not zero or an empty string. Use IS NULL and IS NOT NULL. Comparisons with NULL generally yield UNKNOWN, which does not pass WHERE.

Aggregate functions

```
SELECT DepartmentID,
       COUNT(*) AS EmployeeCount,
       AVG(Salary) AS AverageSalary,
       MIN(Salary) AS MinimumSalary,
       MAX(Salary) AS MaximumSalary
FROM Employee
GROUP BY DepartmentID
HAVING COUNT(*) >= 2;
```

COUNT(*) counts rows. COUNT(column) ignores NULL values. WHERE filters rows before grouping; HAVING filters groups after aggregation.

9. Joins, subqueries, and set operations

```
SELECT e.FullName, d.DepartmentName
FROM Employee AS e
INNER JOIN Department AS d
    ON e.DepartmentID = d.DepartmentID;

SELECT d.DepartmentName, e.FullName
FROM Department AS d
LEFT JOIN Employee AS e
    ON e.DepartmentID = d.DepartmentID;
```

- INNER JOIN returns matching row pairs.
- LEFT JOIN preserves all left-side rows and fills unmatched right-side columns with NULL.
- RIGHT JOIN preserves right-side rows; FULL OUTER JOIN preserves unmatched rows on both sides where supported.
- CROSS JOIN returns the Cartesian product.
- SELF JOIN joins a table to itself, for example employee-manager relationships.

Subqueries

```
SELECT FullName
FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);

SELECT d.DepartmentName
FROM Department d
WHERE EXISTS (
    SELECT 1
    FROM Employee e
    WHERE e.DepartmentID = d.DepartmentID
);
```

Set operations

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers;
```

UNION removes duplicates; UNION ALL preserves duplicates and is often faster. INTERSECT and EXCEPT/MINUS availability and names vary by engine. The queries must return compatible columns.

10. Constraints, views, and indexes

Constraints

```
CREATE TABLE Account (
    AccountID INT PRIMARY KEY,
    Email VARCHAR(200) UNIQUE,
    Balance DECIMAL(12,2) NOT NULL CHECK (Balance >= 0),
    Status VARCHAR(20) DEFAULT 'Active'
);
```

Constraints protect data regardless of which application writes it. A UNIQUE constraint's treatment of NULL values varies across engines.

Views

```
CREATE VIEW PublicEmployee AS
SELECT EmployeeID, FullName, DepartmentID
FROM Employee;
```

A view stores a query definition, not necessarily a separate copy of rows. Materialized views store results and need refresh/maintenance. Views can simplify queries and restrict exposure, but are not a substitute for a full authorization design.

Indexes

```
CREATE INDEX IX_Employee_Department
ON Employee(DepartmentID);

CREATE INDEX IX_Employee_Department_Salary
ON Employee(DepartmentID, Salary);
```

Indexes speed up suitable reads and joins but consume storage and add write/maintenance cost. Column order matters in composite indexes; choose indexes based on actual query patterns and plans.

11. Functional dependencies and normalization

Normalization organizes relations to reduce harmful redundancy and update anomalies. A functional dependency X → Y means values of attribute set X determine values of Y.

Anomalies

• Update anomaly: the same fact must be changed in multiple rows.

• Insertion anomaly: a fact cannot be stored without unrelated data.

• Deletion anomaly: removing one fact unintentionally removes another.

Normal forms

Normal form	Core idea
1NF	Attributes contain atomic values under the chosen model; no repeating groups
2NF	1NF and no non-prime attribute depends on a proper subset of any candidate key
3NF	For every nontrivial $X \rightarrow A$, X is a superkey or A is prime
BCNF	For every nontrivial $X \rightarrow Y$, X is a superkey
4NF	No nontrivial multivalued dependency unless determinant is a superkey
5NF	Join dependencies are implied by candidate keys

Example decomposition

Suppose Enrollment(StudentID, StudentName, CourseID, CourseName, Grade) has key (StudentID, CourseID). If StudentID → StudentName and CourseID → CourseName, names are repeated for each enrollment. Decompose into Student(StudentID, StudentName), Course(CourseID, CourseName), and Enrollment(StudentID, CourseID, Grade).

A good decomposition aims for lossless joins and, where possible, dependency preservation. Normalization is not a goal to apply mechanically: analytical systems sometimes use controlled denormalization after measuring workload needs.

12. Transactions and ACID

A transaction is a logical unit of work. Example: transfer money from one account to another.

Property	Meaning
Atomicity	All operations occur, or none do
Consistency	Constraints remain satisfied when transactions are correct
Isolation	Concurrent transactions are controlled to avoid disallowed interference
Durability	Committed changes survive failures according to the system's guarantees

```
BEGIN TRANSACTION;

UPDATE Account
SET Balance = Balance - 100
WHERE AccountID = 1;

UPDATE Account
SET Balance = Balance + 100
WHERE AccountID = 2;

COMMIT;
-- On an error, application logic may issue ROLLBACK.
```

Transaction syntax varies by database. The application should verify that both accounts exist and that the debit is allowed. Use constraints and appropriate isolation to protect correctness.

13. Concurrency control and locking

Concurrency control preserves correctness when transactions overlap. A schedule is the interleaving of operations from concurrent transactions.

Common anomalies

• Dirty read: read uncommitted data that may later be rolled back.

• Non-repeatable read: reading the same row twice yields different committed values.

• Phantom read: repeating a predicate query yields a different set of rows.

• Lost update: one update overwrites another transaction's change.

Isolation levels

Level	Typical guarantee / trade-off
Read Uncommitted	May allow dirty reads; exact behavior varies
Read Committed	Prevents dirty reads; repeated reads may differ
Repeatable Read	Keeps previously read rows stable; phantom handling varies
Serializable	Strongest standard isolation goal; may reduce concurrency or abort transactions
Snapshot / MVCC modes	Reads from a consistent version; write conflicts still require care

Locking and 2PL

Shared locks typically permit concurrent readers; exclusive locks protect writes. Two-phase locking (2PL) has a growing phase that acquires locks and a shrinking phase that releases locks. Strict 2PL holds write locks until commit/rollback, simplifying recovery. Real engines may also use multiversion concurrency control (MVCC).

14. Deadlocks and schedules

A deadlock occurs when transactions wait in a cycle for resources held by one another.

```
T1: lock(A) -> request(B) -> wait
T2: lock(B) -> request(A) -> wait
```

• Prevention: enforce a global resource order or use a prevention scheme.

• Detection: build a wait-for graph and detect cycles.

• Recovery: abort a victim transaction and retry safely.

• Timeouts can detect prolonged waits but may abort work unnecessarily.

A schedule is conflict-serializable if its precedence graph is acyclic. Conflicting operations access the same data item from different transactions and at least one is a write.

15. Recovery and logging

Recovery returns a database to a valid state after a crash or transaction failure.

• Write-ahead logging (WAL): log records must be durable before corresponding data pages are written, according to the engine's protocol.

• Undo reverses uncommitted effects; redo reapplies committed effects when needed.

• Checkpoints limit how much log must be processed during recovery.

• Backups and transaction logs support restore and point-in-time recovery where configured.

• Replication is not a replacement for backups; accidental deletes may replicate.

Recovery algorithms vary. Follow the database vendor's backup, restore, and durability documentation and test restores regularly.

16. Storage, files, and indexing structures

Physical storage concepts

• Rows are stored in pages/blocks, which are moved between disk and memory.

• Buffer pools cache frequently used pages.

• Heap files store rows without a particular order; other layouts organize rows by key.

• Sequential and random I/O costs influence query plans.

B-tree and hash indexes

B+ tree indexes keep keys ordered and support equality, range scans, and ordered retrieval. Hash indexes are suited to equality lookup but generally not range scans. Exact capabilities depend on the engine.

A clustered index or clustered table organization affects how table rows are physically/logically organized; the meaning differs by DBMS. Too many indexes increase write cost and storage use.

17. Query processing and optimization

A database commonly parses SQL, validates names and types, rewrites the query, estimates candidate plans, chooses a plan, and executes it.

Optimization principles

• Push filters close to data sources to reduce rows early.

• Use selective predicates and indexes where beneficial.

• Join smaller/reduced intermediate results when the optimizer's estimates support it.

• Avoid SELECT * when only a few columns are needed in stable application queries.

• Inspect EXPLAIN or execution plans; use ANALYZE-style commands carefully because some execute the query.

• Keep statistics current and test with realistic data volumes.

An index does not guarantee a faster query. If a query returns much of a table, a sequential scan may be cheaper. Optimizers use estimates, and inaccurate statistics can lead to poor plans.

18. Database security

• Use least privilege: applications should have only required permissions.

• Use parameterized queries to reduce SQL injection risk.

• Separate application, read-only, migration, and administrator credentials.

• Protect backups and replicas as carefully as primary data.

• Encrypt traffic and storage according to risk and platform support.

• Audit sensitive operations and manage credentials securely.

• Patch database software and restrict network access.

• Mask or minimize personal and confidential data in development environments.

```
-- Illustrative SQL; permission syntax differs by database.
GRANT SELECT ON Employee TO reporting_user;
```

Authentication establishes identity; authorization determines allowed actions. Row-level security, roles, auditing, and encryption are platform-specific features.

19. Stored procedures, functions, and triggers

Stored procedures package database-side operations. Functions return values or tables depending on the DBMS. Triggers run automatically in response to configured events.

```
-- Illustrative trigger syntax varies significantly by DBMS.
CREATE TRIGGER example_trigger
AFTER INSERT ON Employee
FOR EACH ROW
-- database-specific trigger body
;
```

Use stored logic when it improves consistency, performance, or operational control. Triggers can hide side effects and complicate debugging; document them and test their transaction behavior.

20. Distributed databases and replication

A distributed database stores or processes data across multiple nodes.

• Replication maintains copies across nodes for availability or read scaling.

• Sharding/partitioning splits data across nodes, often by key.

• Synchronous replication can strengthen acknowledgement guarantees but increase latency.

• Asynchronous replication can lag and may lose recent acknowledged data under some failure scenarios.

• Distributed transactions coordinate changes across resources and are more complex than local transactions.

CAP theorem

During a network partition, a distributed data system cannot simultaneously guarantee both every request receives a response (availability) and all responses reflect one single-copy-consistent state (strong consistency) for the affected operation model. CAP does not mean a system simply picks any two in all circumstances; the trade-off is specifically under partition.

Consistency concepts

Strong consistency aims for reads to reflect the latest successful write under defined conditions. Eventual consistency allows replicas to converge if updates stop and communication recovers. Choose based on correctness needs, not slogans.

21. NoSQL databases

Type	Typical use	Example concept
Key-value	Fast lookup by key, sessions, cache	key → value
Document	Flexible records and nested data	JSON-like document
Wide-column	Large distributed datasets and high write volume	partition key + columns
Graph	Relationship-heavy traversal	nodes + edges
Time-series	Metrics, telemetry, events	timestamped measurements

NoSQL is a broad family, not a single data model. Many systems offer transactions, indexes, and query languages, but their consistency, joins, schema rules, and scaling approaches differ.

SQL versus NoSQL

Relational databases are a strong default for structured data with complex relationships and transactional constraints. NoSQL may fit flexible document structures, graph traversal, specialized write patterns, or distributed scaling needs. Choose according to access patterns, consistency, operations, and team expertise.

22. Data warehousing, OLAP, and analytics

Operational systems (OLTP) support frequent transactional inserts and updates. Data warehouses support analytics over integrated historical data.

Concept	Meaning
OLTP	Online Transaction Processing; many short transactions
OLAP	Online Analytical Processing; aggregations and multidimensional analysis
Fact table	Measurements such as sales amount
Dimension table	Descriptive context such as product, date, or region
Star schema	Fact table connected to denormalized dimensions
ETL / ELT	Extract, transform, load / extract, load, transform

A slowly changing dimension tracks historical changes in descriptive attributes. Partitioning and column-oriented storage can improve analytical scans.

23. Database design workflow

• Gather requirements, entities, business rules, data volumes, and access patterns.

• Draw an ER model and agree on identifiers and cardinalities.

• Map entities and relationships to tables, keys, and constraints.

• Normalize to reduce anomalies; document any intentional denormalization.

• Choose types, indexes, naming conventions, and delete/update behavior.

• Write migrations and seed/test data.

• Test correctness, concurrency, security, and query performance.

• Plan backup, monitoring, upgrades, and restore procedures.

Example schema

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL
);

CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    Title VARCHAR(150) NOT NULL
);

CREATE TABLE Enrollment (
    StudentID INT NOT NULL,
    CourseID INT NOT NULL,
    EnrolledOn DATE NOT NULL,
    Grade VARCHAR(2),
    PRIMARY KEY (StudentID, CourseID),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```

24. Performance and operational best practices

• Measure slow queries using logs and execution plans.

• Use connection pooling where supported; close/dispose connections promptly.

• Keep transactions short to reduce lock duration and contention.

• Batch writes when safe and supported.

• Use pagination with stable ordering for large result sets.

• Avoid N+1 query patterns in application code.

• Monitor CPU, memory, I/O, locks, cache hit rates, replication lag, and storage.

• Test backups by restoring them to a separate environment.

• Use schema migration scripts with review and rollback plans.

Pagination

```
-- Common offset pagination; syntax broadly portable
SELECT EmployeeID, FullName
FROM Employee
ORDER BY EmployeeID
OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;
```

Pagination syntax differs across engines. For very large tables, keyset pagination (filtering after the last seen ordered key) can be more efficient than large offsets.

25. Common interview questions

Q. What is a DBMS?
Software that manages database definition, access, integrity, concurrency, security, and recovery.

Q. What is the difference between DBMS and RDBMS?
RDBMS refers to a DBMS based on the relational model; DBMS is the broader category.

Q. Primary key versus unique key?
A primary key is the chosen main row identifier and is non-null; UNIQUE constraints enforce uniqueness with NULL semantics that vary by engine.

Q. DELETE versus TRUNCATE versus DROP?
DELETE removes selected rows; TRUNCATE removes all rows efficiently where supported; DROP removes the object definition and its data.

Q. WHERE versus HAVING?
WHERE filters rows before grouping; HAVING filters groups after aggregation.

Q. INNER JOIN versus LEFT JOIN?
INNER returns matches; LEFT preserves all left-side rows.

Q. What is normalization?
Structuring relations to reduce redundancy and modification anomalies.

Q. What is a transaction?
A unit of work governed by commit/rollback and transaction guarantees.

Q. What is an index?
An auxiliary structure that can speed reads at storage and write-maintenance cost.

Q. What is a deadlock?
A cycle of transactions waiting on resources held by one another.

Q. What is a view?
A named query that presents data as a virtual table; materialized views store results.

Q. What is SQL injection?
A vulnerability where untrusted input changes query structure; parameterized queries help prevent it.

26. Practice SQL schema and queries

```
-- Tables: Student(StudentID, FullName)
-- Course(CourseID, Title)
-- Enrollment(StudentID, CourseID, Grade)

-- 1. List all students
SELECT StudentID, FullName FROM Student;

-- 2. Count enrollments per course
SELECT c.CourseID, c.Title, COUNT(e.StudentID) AS Enrolled
FROM Course c
LEFT JOIN Enrollment e ON e.CourseID = c.CourseID
GROUP BY c.CourseID, c.Title;

-- 3. Students enrolled in at least one course
SELECT DISTINCT s.StudentID, s.FullName
FROM Student s
JOIN Enrollment e ON e.StudentID = s.StudentID;

-- 4. Students not enrolled in any course
SELECT s.StudentID, s.FullName
FROM Student s
WHERE NOT EXISTS (
  SELECT 1 FROM Enrollment e
  WHERE e.StudentID = s.StudentID
);

-- 5. Find courses with no enrollments
SELECT c.CourseID, c.Title
FROM Course c
WHERE NOT EXISTS (
  SELECT 1 FROM Enrollment e
  WHERE e.CourseID = c.CourseID
);
```

27. Exercises and mini-projects

Theory exercises

• Draw an ER diagram for a library with books, members, and loans.

• Identify primary, candidate, and foreign keys for a college database.

• Normalize a sales table to 3NF and explain each decomposition.

• Explain dirty reads, non-repeatable reads, and phantom reads with schedules.

• Compare B+ tree and hash indexes for equality and range queries.

• Explain the CAP trade-off during a network partition.

SQL exercises

• Find the second-highest distinct salary.

• Find departments with more than five employees.

• List employees whose salary is above their department average.

• Show customers who have never placed an order.

• Use GROUP BY to calculate monthly sales totals.

• Create an index based on a realistic filter-and-sort query and compare plans.

Mini-project ideas

• Library management system with members, books, loans, and overdue reports.

• College database with students, courses, instructors, and enrollments.

• Inventory and sales database with stock constraints and transaction-safe checkout.

• Help-desk ticket system with status history, assignment, and reporting.

• Analytics warehouse with fact and dimension tables.

28. Quick revision sheet

Topic	Remember
Keys	Primary identifies; foreign references; candidate is minimal unique key
ER mapping	1:N foreign key on N side; M:N junction table
SQL order (conceptual)	FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
NULL	Use IS NULL; NULL is not equal to zero or empty text
Normalization	1NF atomic values; 2NF no partial dependency; 3NF remove transitive dependency under formal conditions; BCNF every determinant is a
ACID	Atomicity, Consistency, Isolation, Durability
Indexes	Improve some reads; cost storage and writes
Concurrency	Isolation levels trade consistency guarantees against concurrency
Recovery	WAL, undo/redo, checkpoints, backups
Security	Least privilege, parameterized SQL, encryption, auditing
Operations	Monitor, patch, back up, and test restores

Learning roadmap

• Stage 1: tables, keys, constraints, SELECT, filtering, sorting.

• Stage 2: joins, aggregates, subqueries, views, indexes.

• Stage 3: ER modeling, functional dependencies, normalization.

• Stage 4: transactions, isolation, locks, deadlocks, recovery.

• Stage 5: query plans, tuning, security, backup and restore.

• Stage 6: distributed systems, NoSQL, warehousing, and production operations.

Note: DBMS concepts are broadly shared, but SQL syntax, isolation behavior, indexing options, procedural languages, and operational commands vary by product. Use the official documentation for the database engine you practice with.